SAP Security Expert Curriculum

Introduction to Cryptography

Session 2: Symmetric Encryption

Overall Agenda for Introduction to Cryptography

Session 1: Introduction and Crypto Basics

Session 2: Symmetric Encryption

Session 3: Asymmetric Encryption

Agenda for Session 2: Symmetric Encryption

Symmetric Encryption: Motivation, Block Ciphers (DES, AES)

Modes of operation, stream ciphers

Cryptographic Hash Functions

Demo: Collision for SHA1

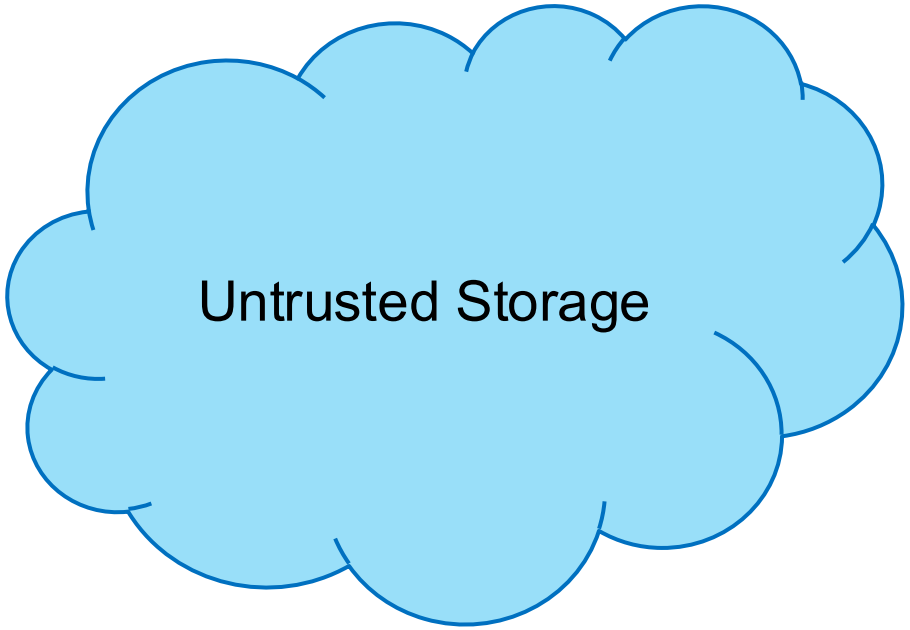
Cryptographic Hash Functions: Integrity via Message Authentication Codes

Authentication via Galois/Counter Mode

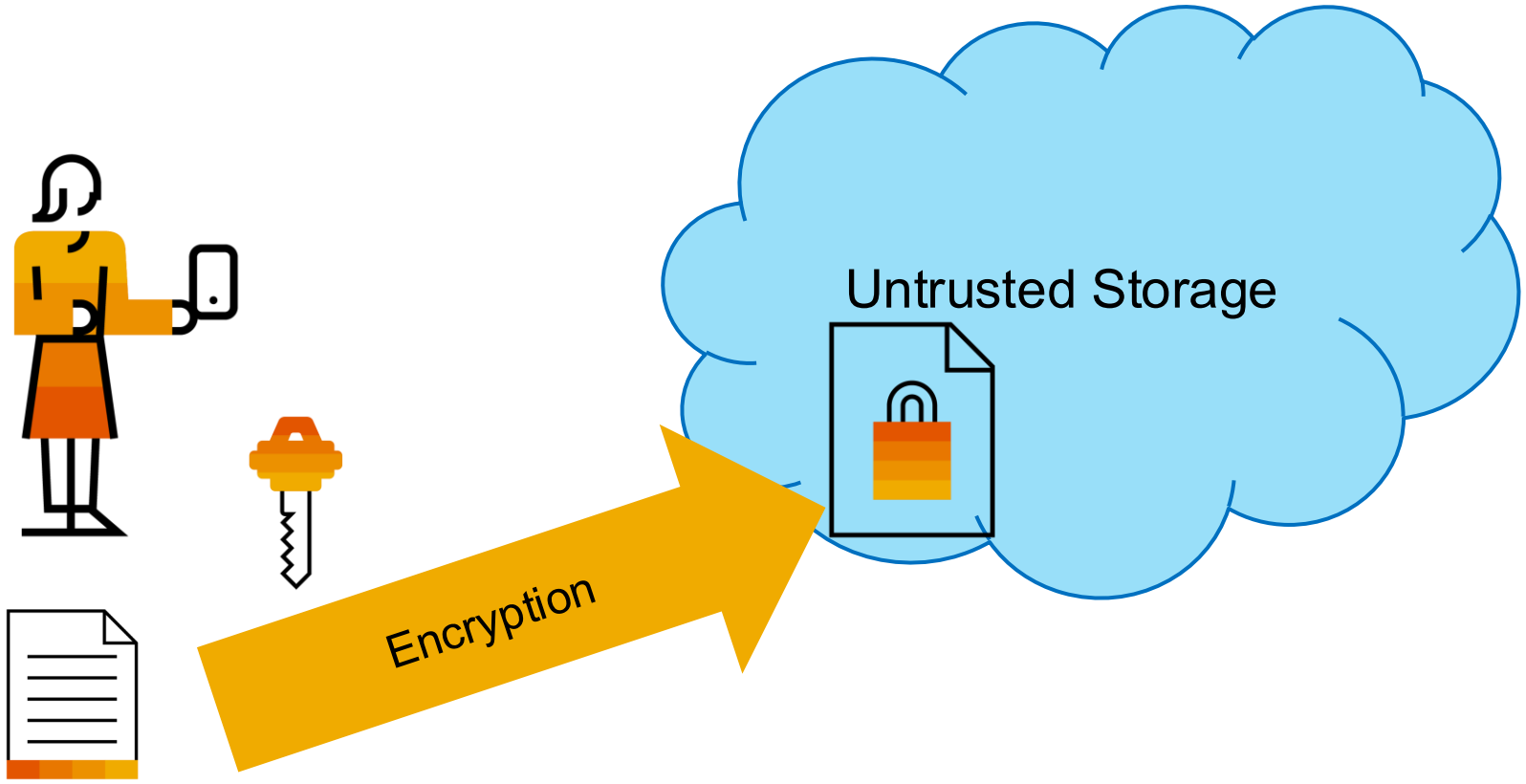
Passwords and Keys: Derivation Functions and Password Stores

Motivation

Symmetric Encryption



Symmetric Encryption



Symmetric Encryption

Symmetric Encryption

Symmetric Key Encryption

Key Generation:

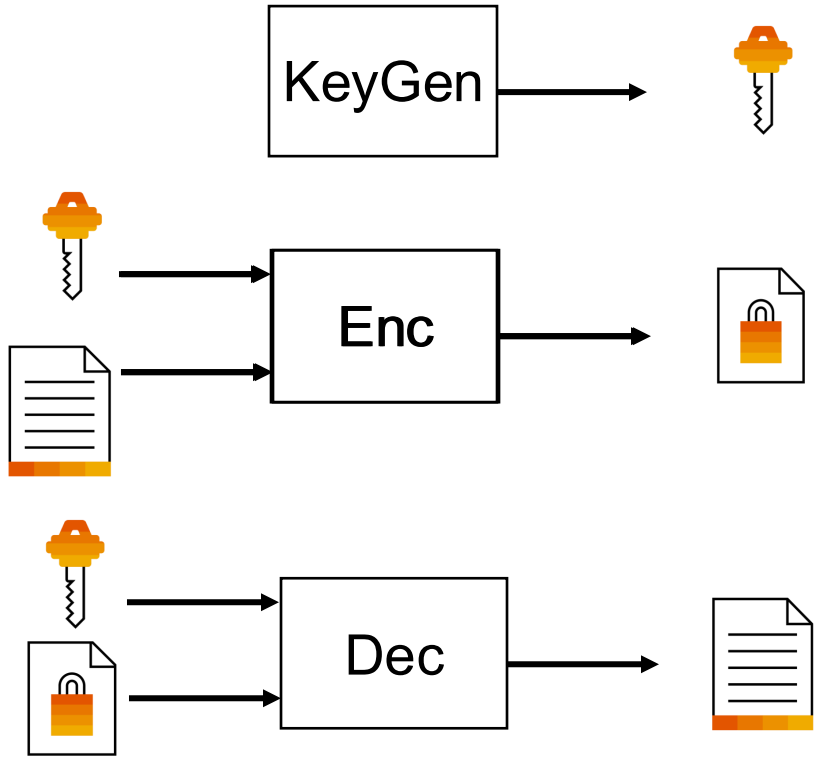
- Generates a key used for encryption and decryption.

Encrypt:

- Given the key and a plaintext outputs a ciphertext.

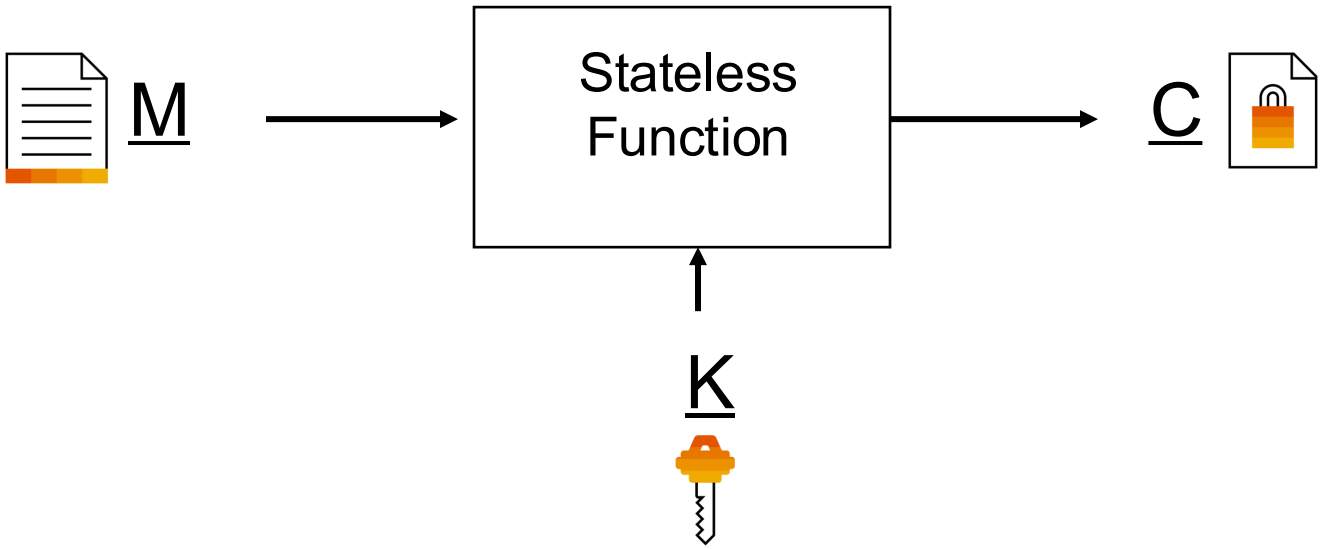
Decrypt:

- Given the key and a ciphertext outputs the corresponding plaintext.



Block Ciphers

Block Ciphers



M: Plaintext block with bit size n
C: Ciphertext block with bit size n
K: Key with bit size s

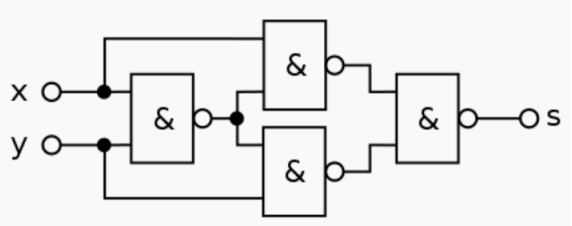
Data Encryption Standard (DES)

Introduction Exclusive OR - XOR

- boolean binary operation
- easy to apply and remove
- A = 1001 1110
- B = 0111 0001
- A XOR B = C = 1110 1111

- Result of C XOR B = ?
- Result of C XOR A = ?

x	y	s
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**0**



XOR build from NAND gates

Data Encryption Standard (DES)

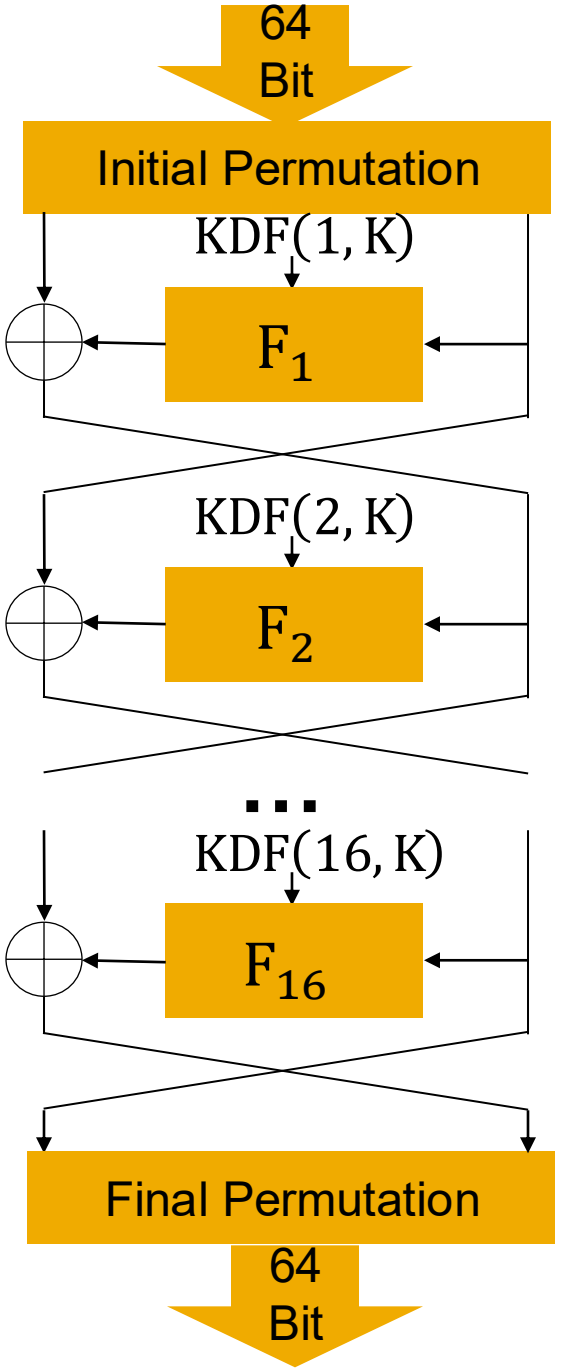
Standardized in 1977

Key size: 56 bits

Block size: 64 bits

Structure: Feistel Network

- 16 Rounds with different round keys



Data Encryption Standard (DES) - Decryption

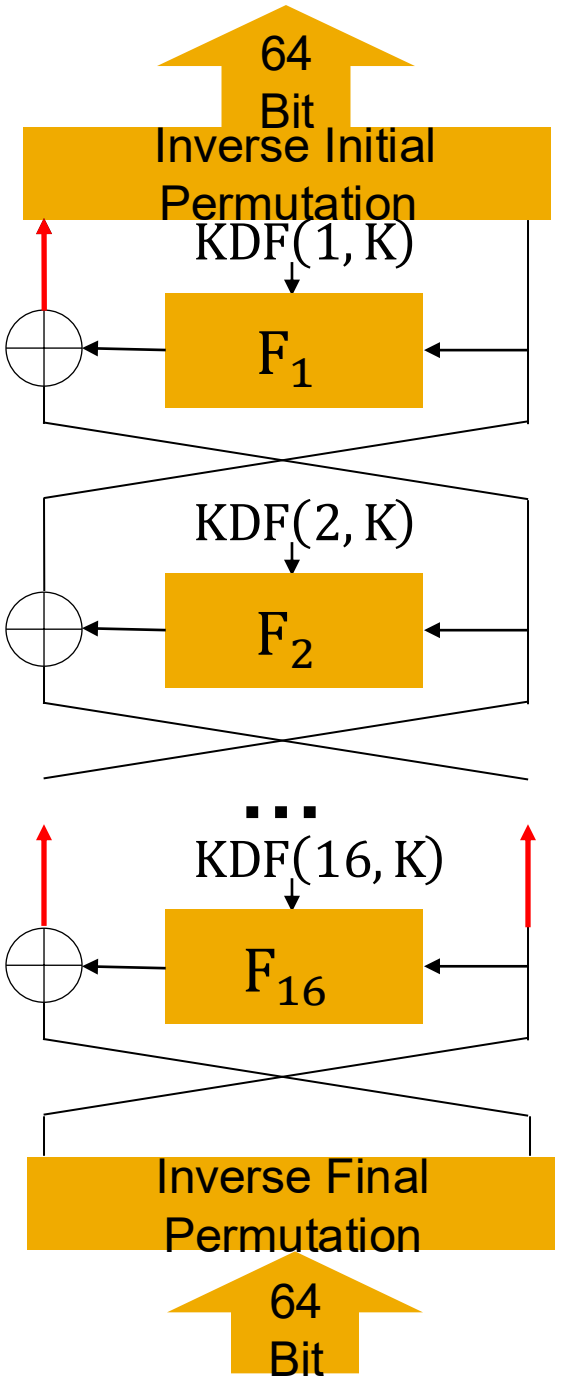
Standardized in 1977

Key size: 56 bits

Block size: 64 bits

Structure: Feistel Network

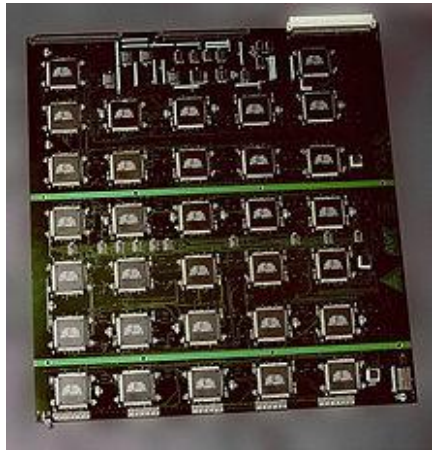
- 16 Rounds with different round keys

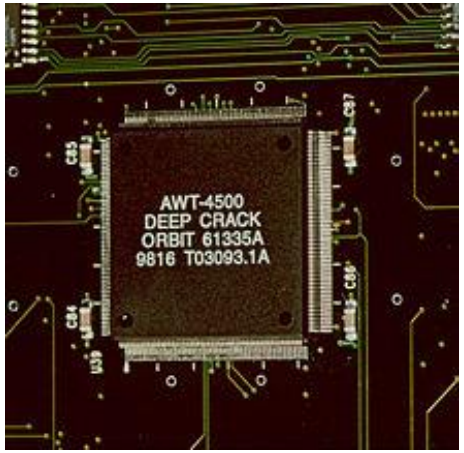


More Algorithms based on Feistel Networks:

- Blowfish & Twofish
- Mars
- RC6

Data Encryption Standard (DES) – Too weak





DES should not be used since key space is too small

Deep Crack 1998

29 circuit boards with 64 ASICs each – 250.000 $

brute force all keys in 9 days

COPACOBANA 2006

120 FPGAs, 6,4 days for whole key space, 10.000$

ASIC = Application-specific integrated circuit (custom chips)

FPGA = Field Programmable Gate Array („programmable circuit board“)

Advanced Encryption Standard (AES)

Advanced Encryption Standard (AES)

Standardized in 2001 by a NIST competition (Rijndael Cipher became AES)

winner	finalist	round 1	candidate	
yes	yes	yes	Rijndael	Vincent Rijmen, Joan Daemen
	yes	yes	MARS	Carolynn Burwick, Don Coppersmith, Edward
	yes	yes	RC6	Ron Rivest, Matt Robshaw, Ray Sidney, Yiqun
	yes	yes	Serpent	Ross Anderson, Eli Biham, Lars Knudsen
	yes	yes	Twofish	Bruce Schneier, John Kelsey, Doug Whiting, D

Advanced Encryption Standard (AES)

Requirements of the competition:

Block Size: Must support 128-bit blocks.

Key Sizes: Must support at least 128-bit, 192-bit, and 256-bit keys.

Security: Resistant to cryptanalysis and mathematical weaknesses.

Efficiency: Works well on 8-bit, 32-bit, and high-performance systems.

Flexibility: Adaptable to different hardware/software implementations.

Simplicity: Easy to implement with minimal resource use.

Advanced Encryption Standard (AES)

Evaluation Criteria of the competition:

Security: Resistance to attacks (differential, linear, algebraic, etc.).

Performance: Speed on various platforms.

Efficiency: Low memory and processing requirements.

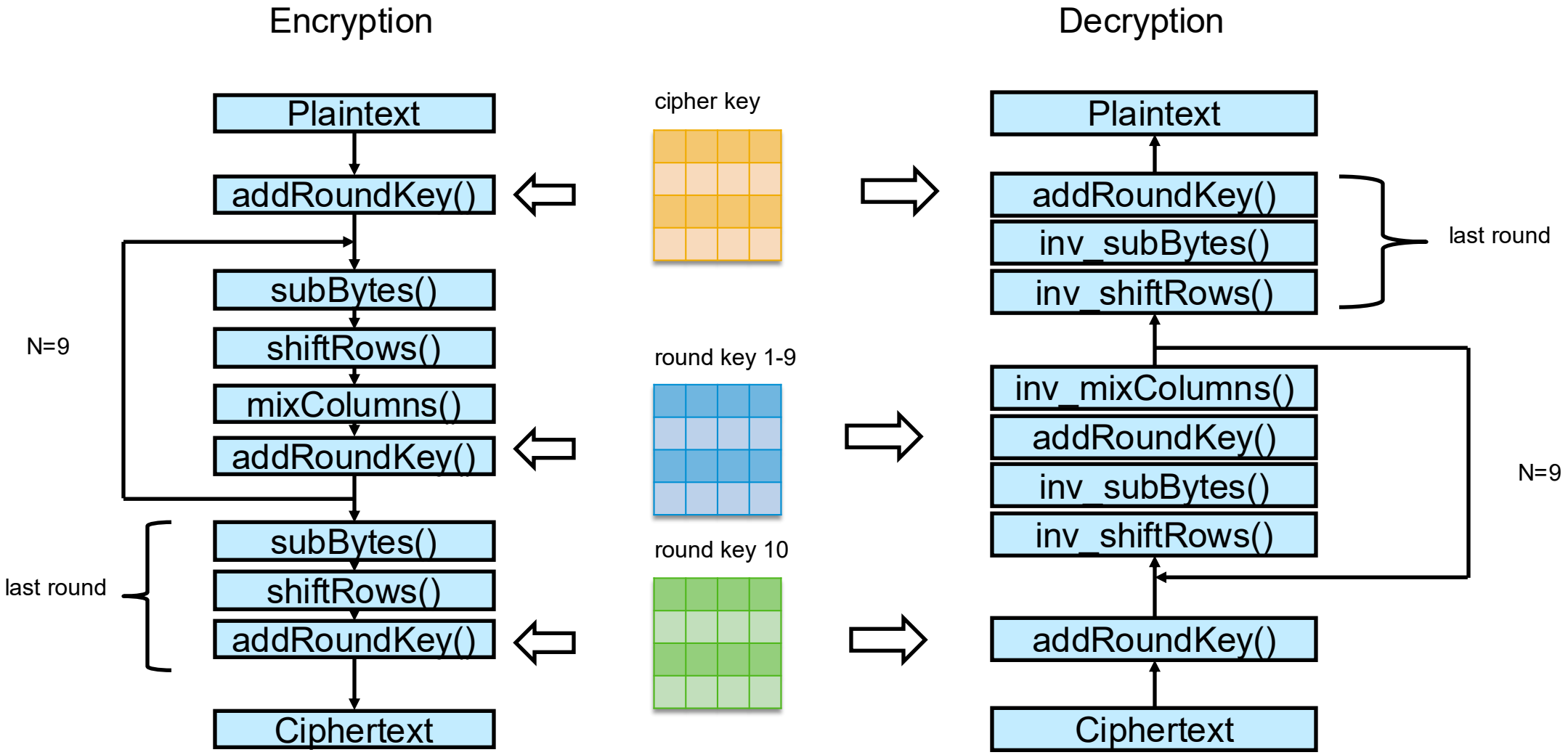
Complexity: Ease of implementation and reduced error risk.

Flexibility: Works on multiple architectures and use cases.

Advanced Encryption Standard (AES)
Encryption Steps

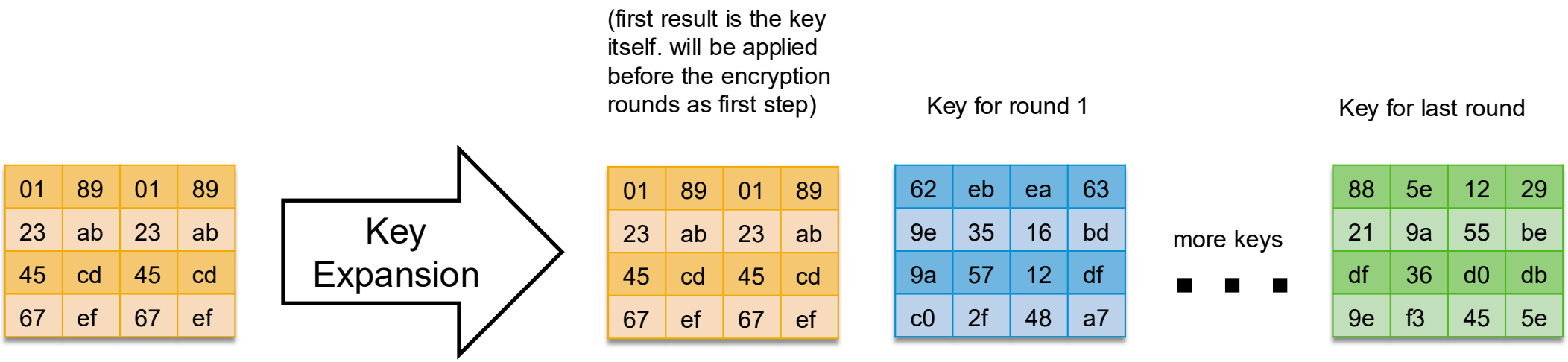
1. Key expansion – generation of one key per round
2. Encryption round
    1. SubBytes() – S-Box substitution
    2. ShiftRows() – Transposition of Matrix rows
    3. MixColumns() – Mix bytes of columns
    4. AddRoundKey() – Add the specific round key from Key Expansion

Overview AES-128



Advanced Encryption Standard (AES)
1. Key Expansion

- 128 bit key in hex: **0x0123456789abcdef0123456789abcdef**
- inside the algorithm everything is calculated in 4x4 bytes matrices
- we generate 1 key more than rounds

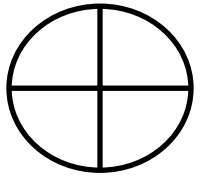


Advanced Encryption Standard (AES)
2. Encryption Rounds – initial round key before rounds start

- 128 bit key in hex: **0x0123456789abcdef0123456789abcdef**
- Plaintext: „Hello World!!!!!“ -> **0x48656c6c6f20576f726c642121212121**

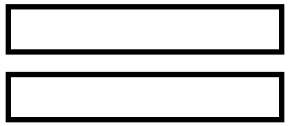
Plaintext hex in 4x4 Matrix

48	6f	72	21
65	20	6c	21
6c	57	64	21
6c	6f	21	21



(first result of the key expansion. will be applied before the encryption rounds as first step)

01	89	01	89
23	ab	23	ab
45	cd	45	cd
67	ef	67	ef



Plaintext xored with first key

addRoundKey()

49	e6	73	a8
46	8b	4f	8a
29	9a	21	ec
0b	80	46	ce

Advanced Encryption Standard (AES)
3. Encryption Rounds – subBytes()

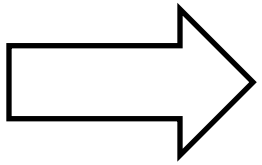
- substitution of every single byte with a given lookup table called s-box (there is also an Inverse x-box to reverse this step)

Example for 49 -> 3b.

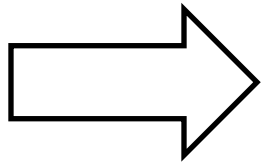
Rows for the first nibble (4 bits) and columns for second nibble

current state

49	e6	73	a8
46	8b	4f	8a
29	9a	21	ec
0b	80	46	ce



	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

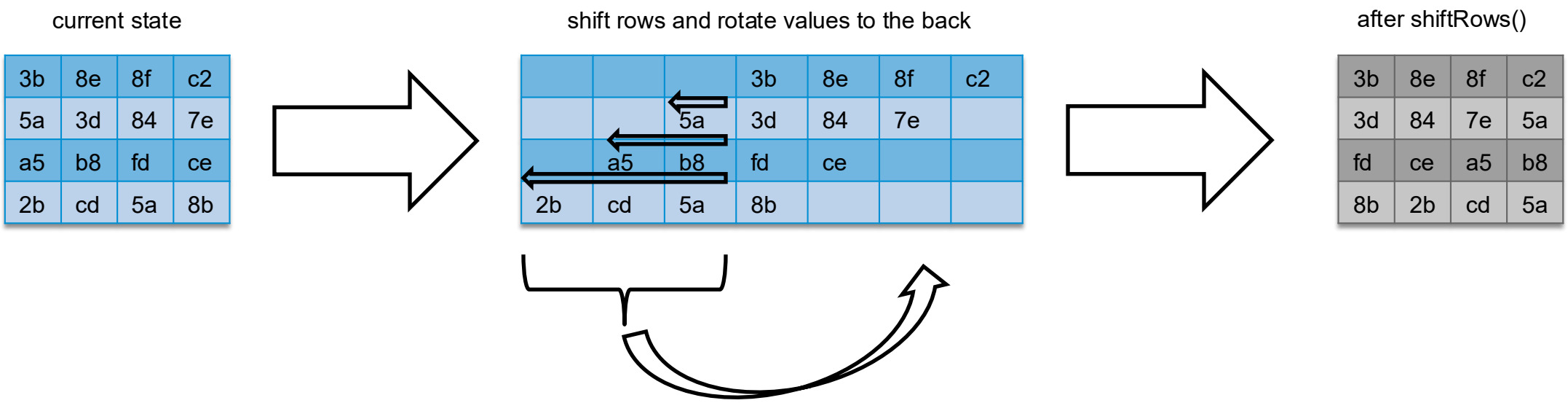


after subBytes()

3b	8e	8f	c2
5a	3d	84	7e
a5	b8	fd	ce
2b	cd	5a	8b

Advanced Encryption Standard (AES)
4. Encryption Rounds – shiftRows()

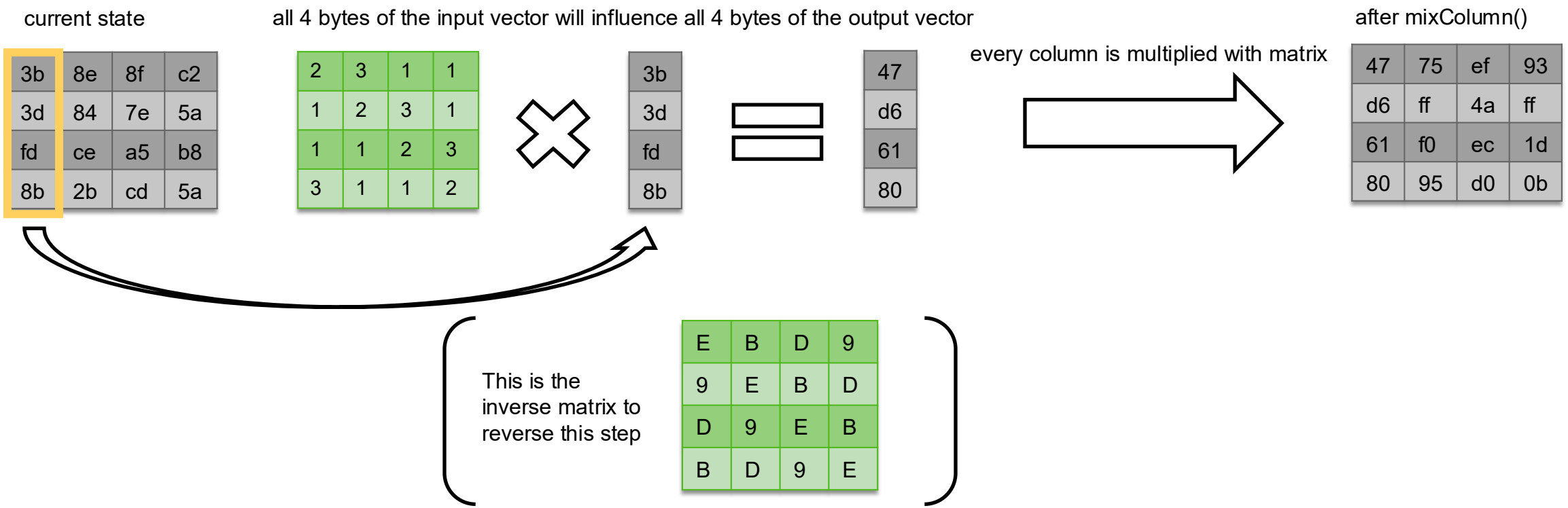
- shifting/ rotation of rows



this is easy to reverse – we just to need to shift the other way arround

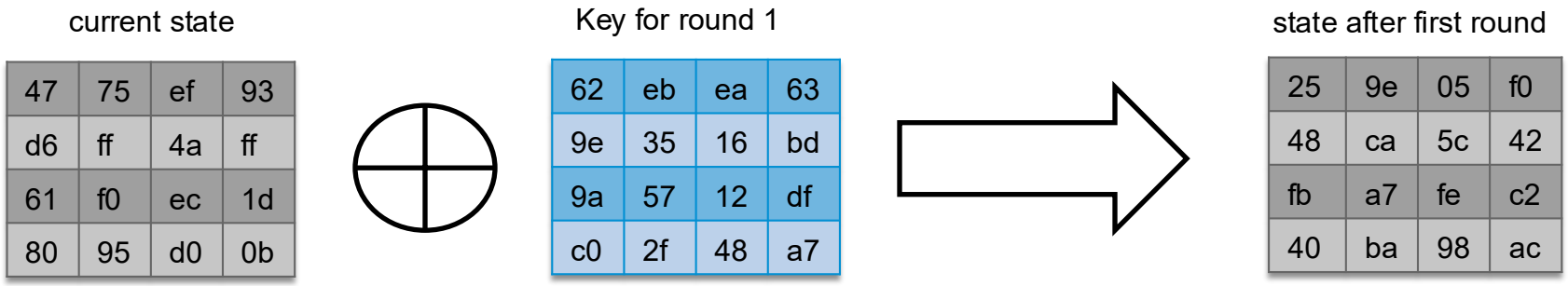
Advanced Encryption Standard (AES)
5. Encryption Rounds – mixColumns()

- mix the values of each column
- Vector * Matrix multiplication for efficiency
- this is not your normal multiplication, calculation based on a Galois Finite Field GF(2^8)



Advanced Encryption Standard (AES)
6. Encryption Rounds – addRoundKey

- remember the Key Expansion?
- At the end of every round we xor the current roundkey to our state

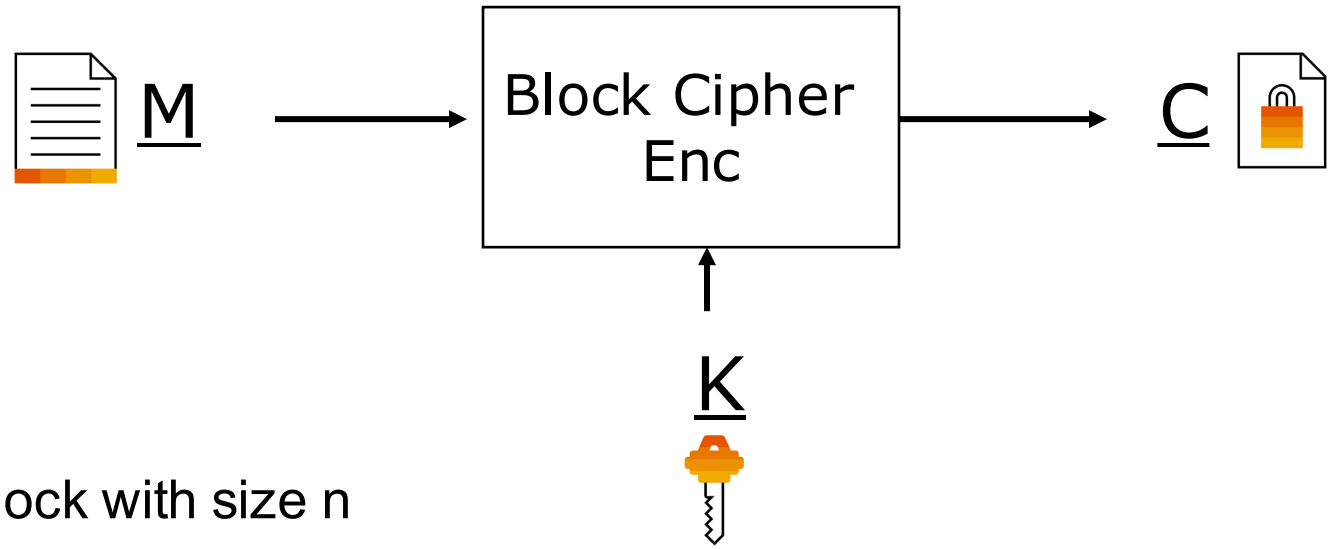


XOR is self inverse. XORing with the round key again will reverse the operation

And now we repeat these 4 steps for at least 9 more rounds in AES-128

Modes of Operations

Modes of Operations

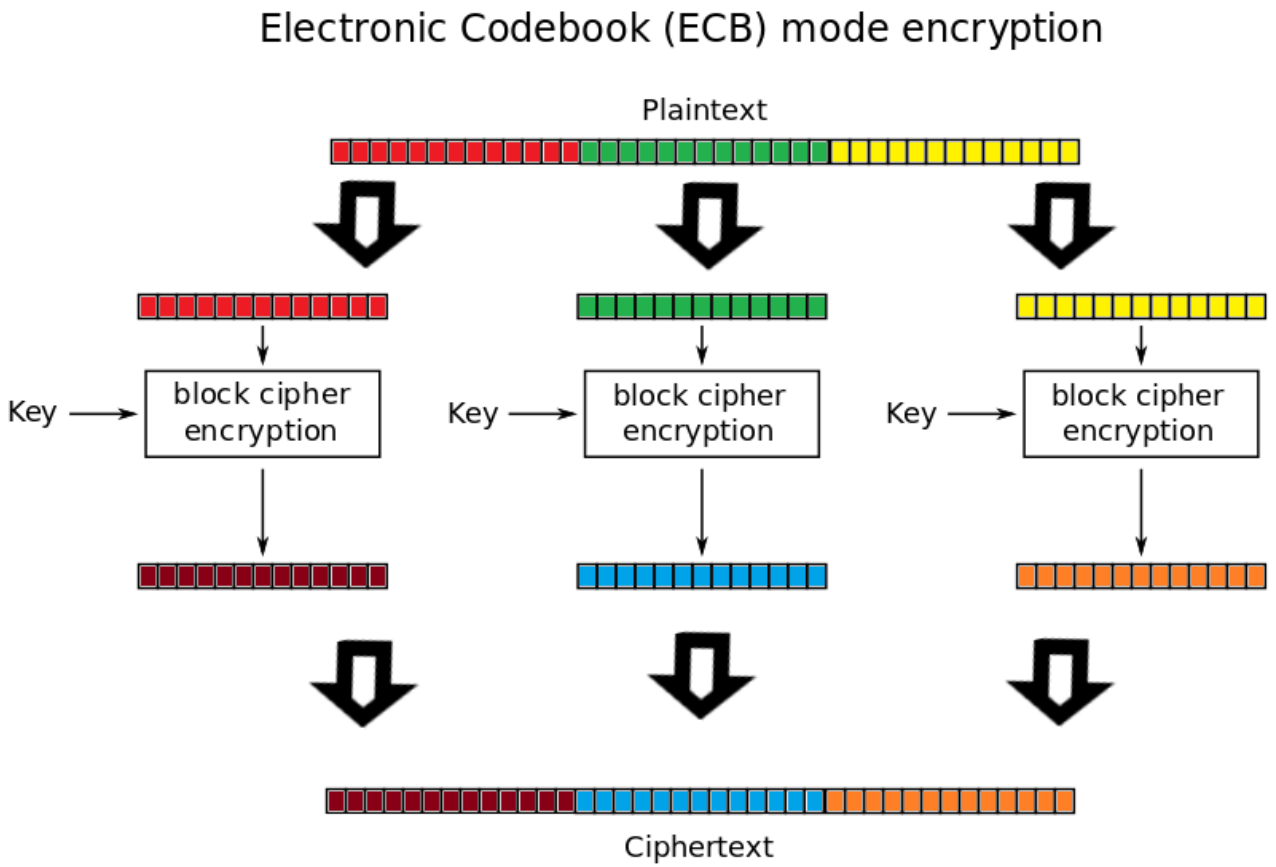


M: Plaintext block with size n

How do you encrypt data D with bit size greater than n?

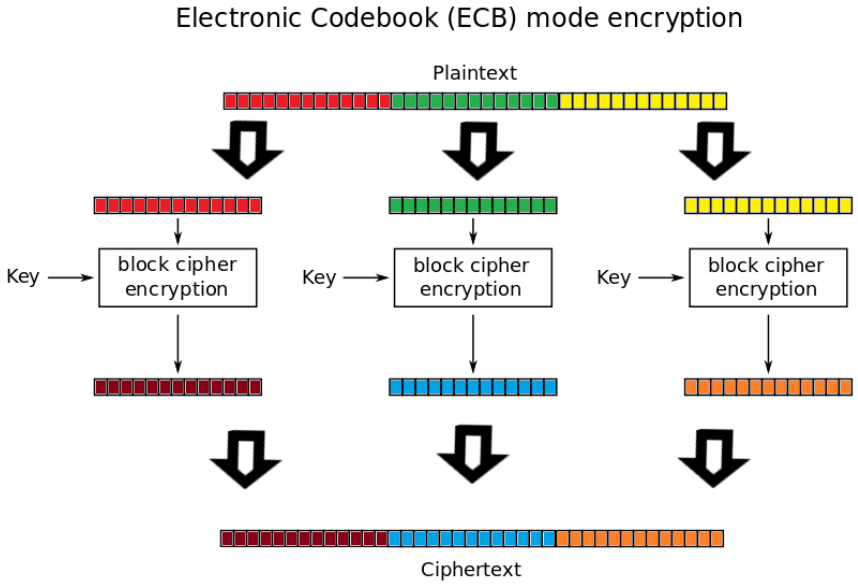
Electronic Codebook Mode (ECB)

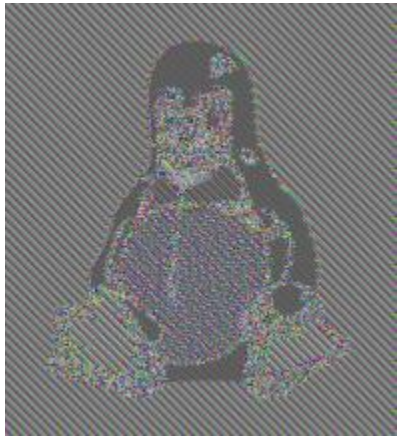
Initial **Bad** Idea: Divide data D into blocks and encrypt each block independently.



Electronic Codebook Mode (ECB)

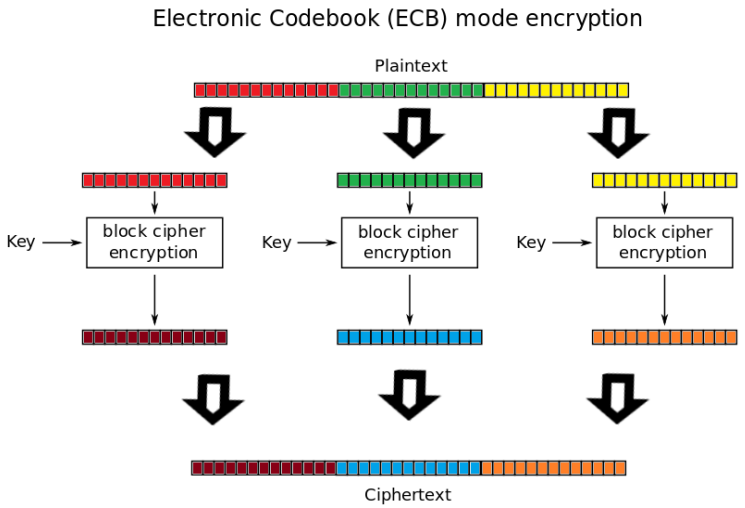
Initial **Bad** Idea: Divide data D into blocks and encrypt each block independently.

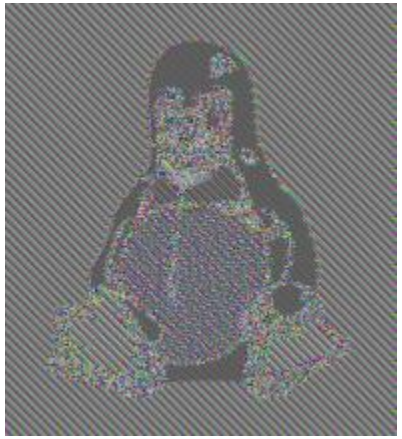




Electronic Codebook Mode (ECB)

Initial **Bad** Idea: Divide data D into blocks and encrypt each block independently.

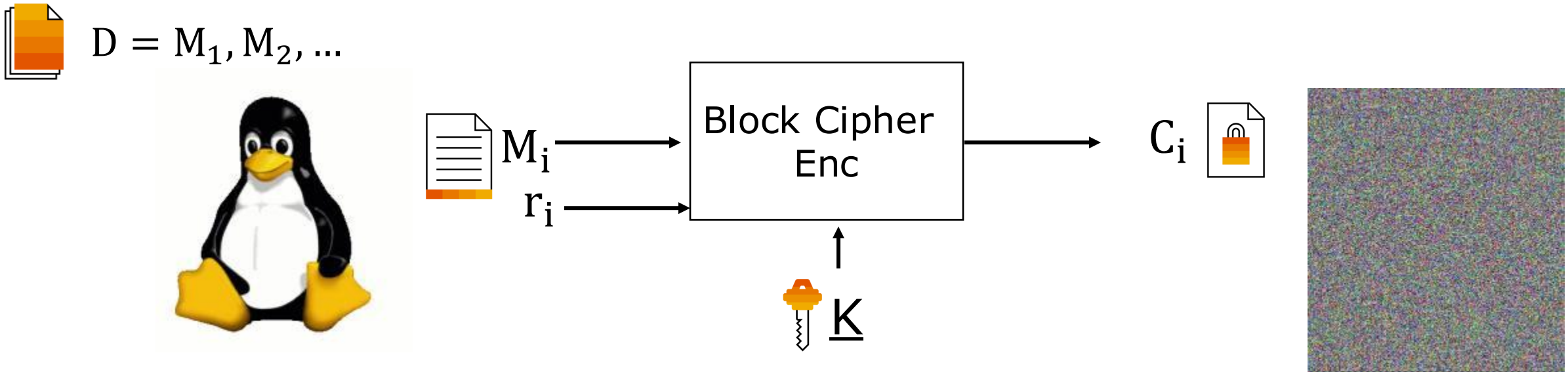




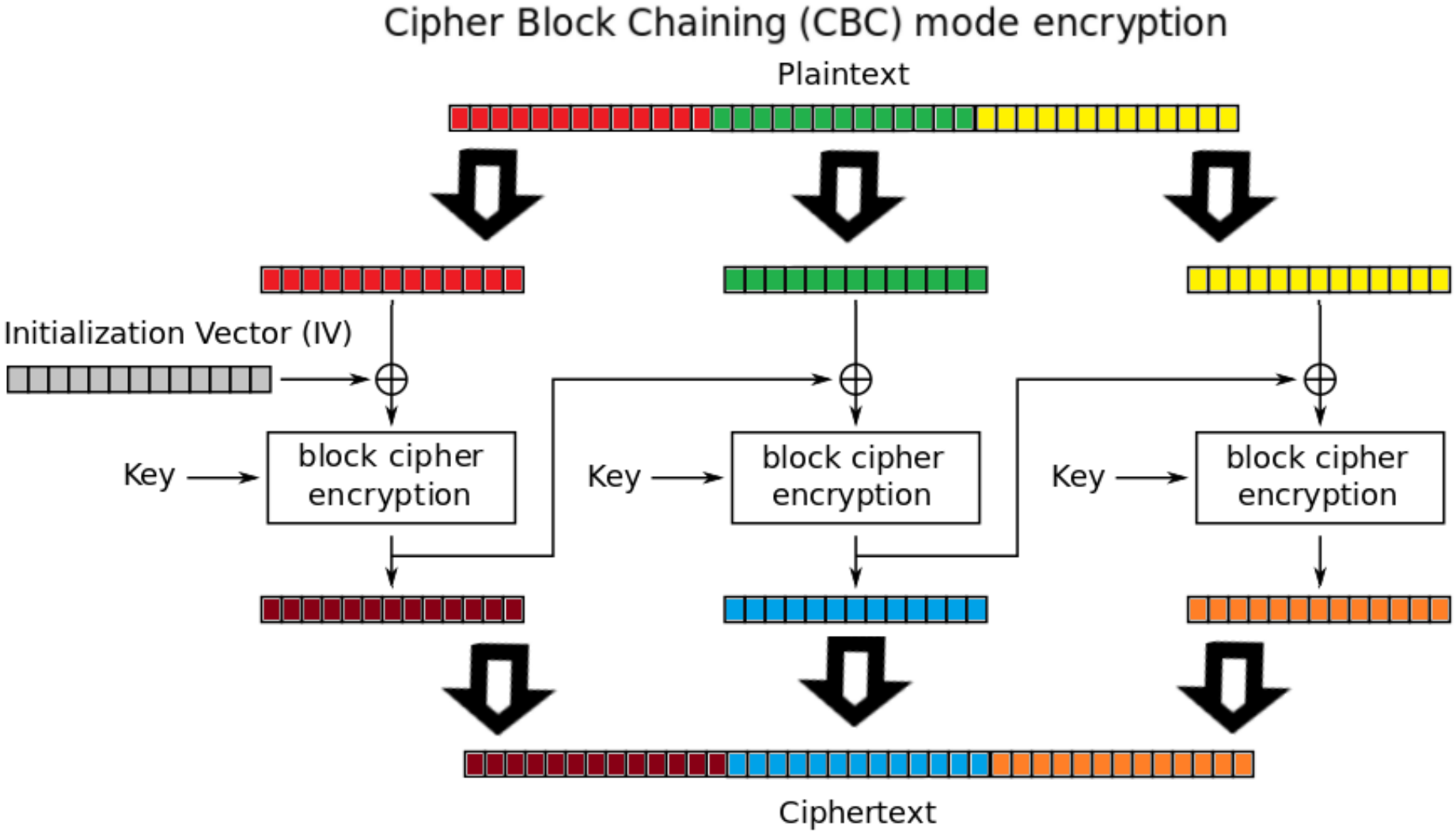
- Dictionary Attack
  - Same plaintext block results in same ciphertext block
- Block ordering can be modified
- Blocks can be injected
- Blocks can be doubled
- Blocks can be deleted

Randomized Encryption

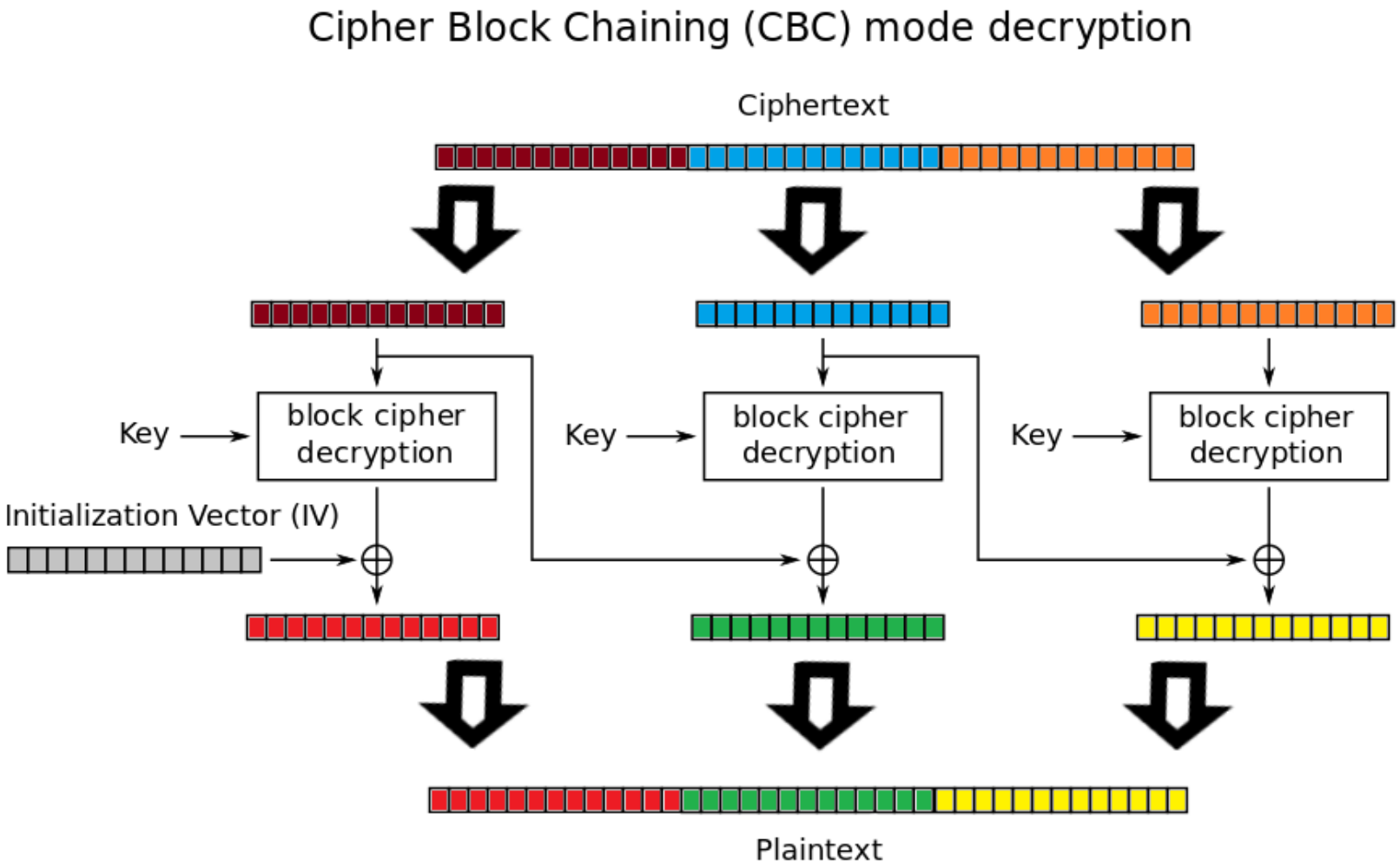
Better: Add random vector $R = r_1, r_2, \ldots$ to prevent dictionary attacks.



Cipher Block Chaining (CBC)



Cipher Block Chaining (CBC) - Decryption



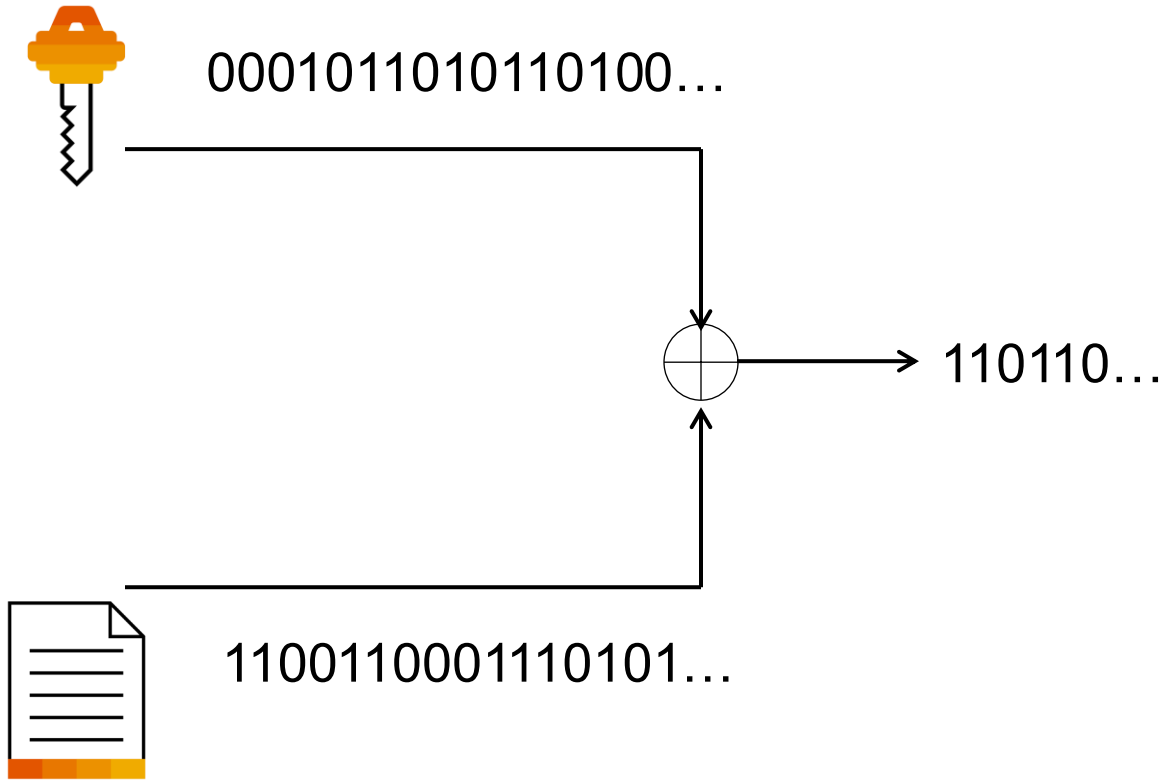
Stream Ciphers

One Time Pad (the idea behind stream ciphers)

Cannot be cracked if:

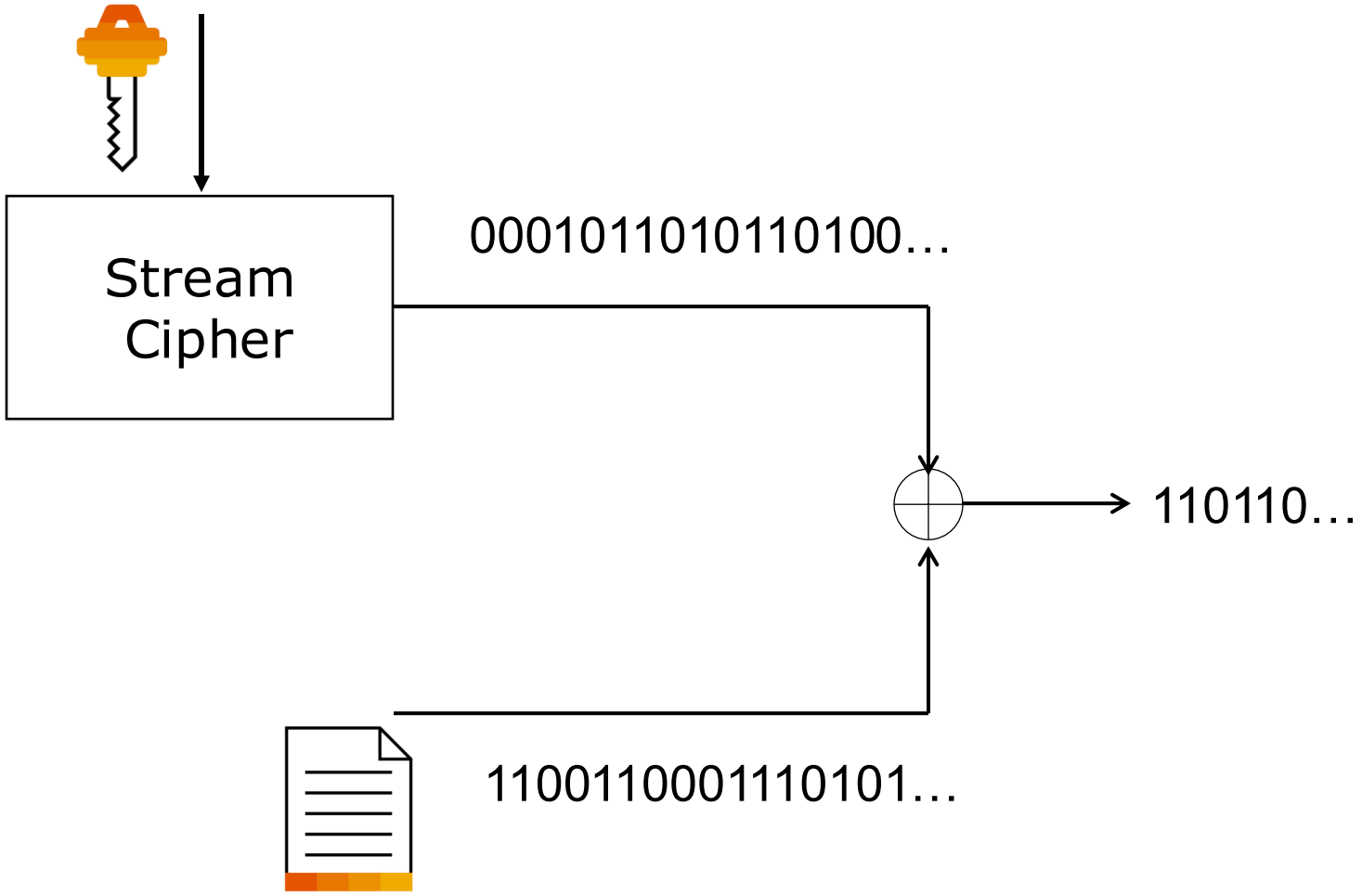
- Key is never re-used (in whole or in part)
- Key is kept completely secret
- Key is at least as long as plaintext
- Key is truly random

Idea: Generate a bit sequence that is looking as good as random but has a practically short seed.



Stream Ciphers

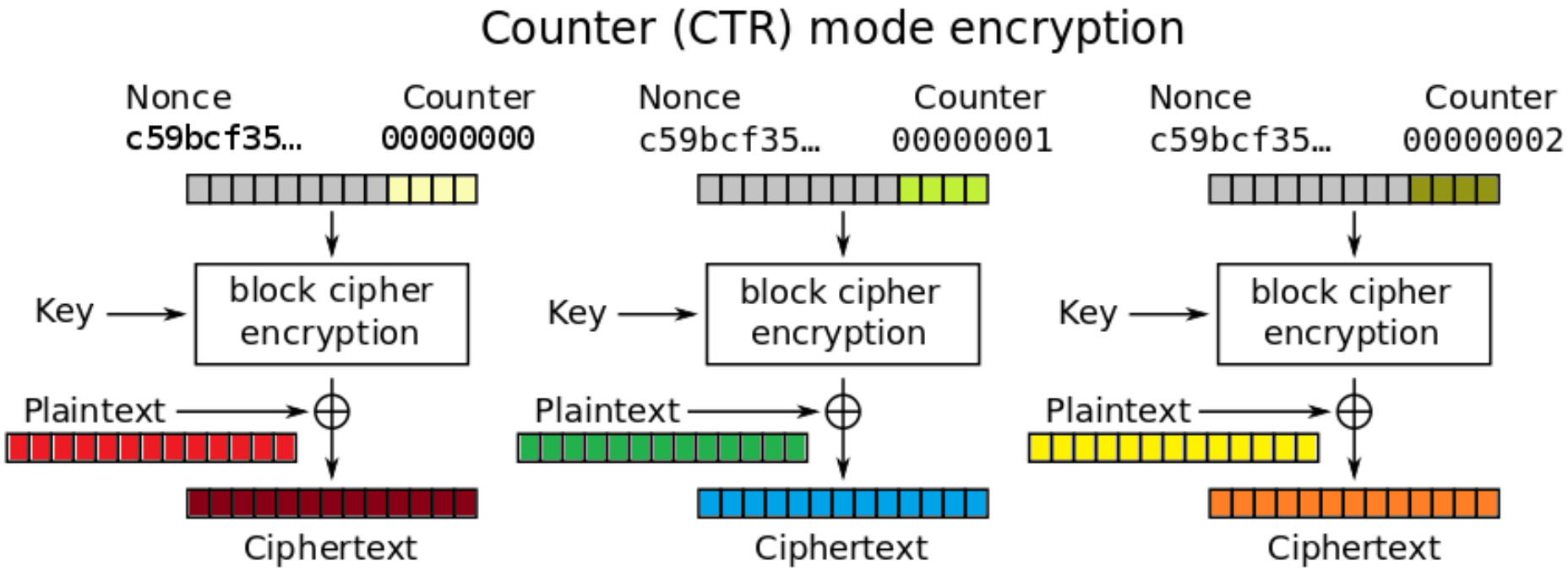
Modern Stream ciphers require a nonce as additional input.



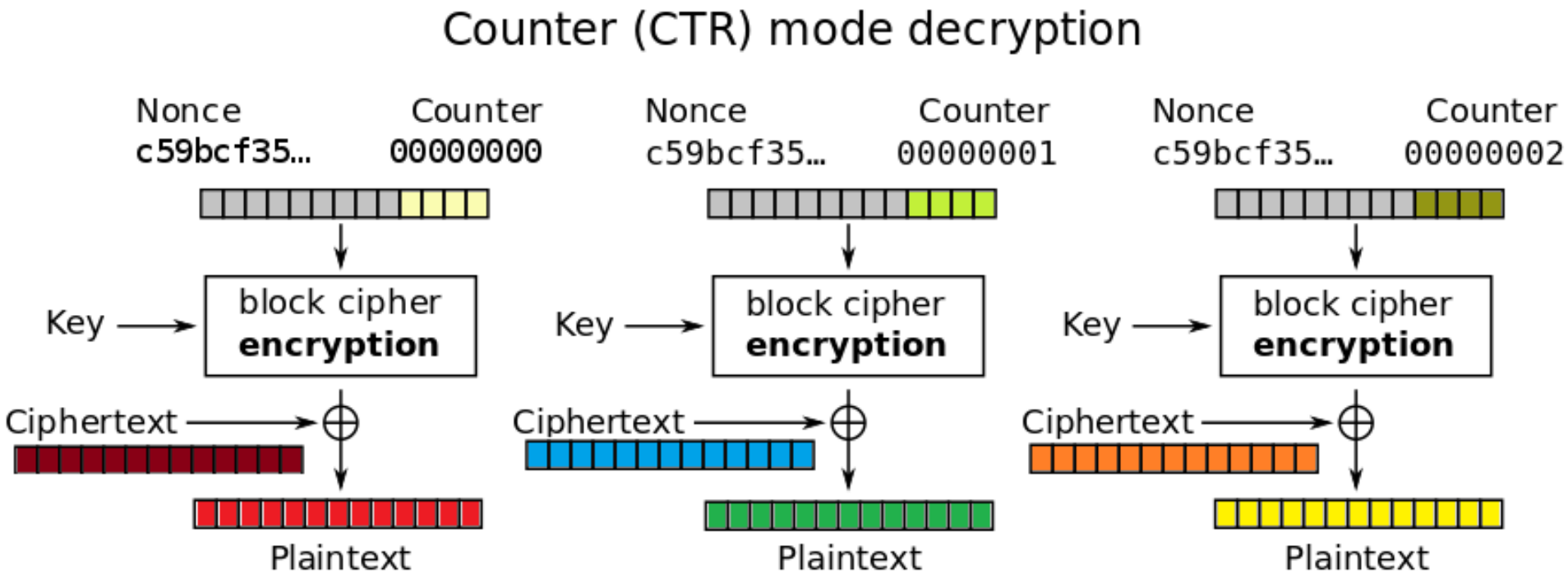
Examples:

- ChaCha20
- RC4 (not secure anymore)

CTR (Counter mode) Encryption



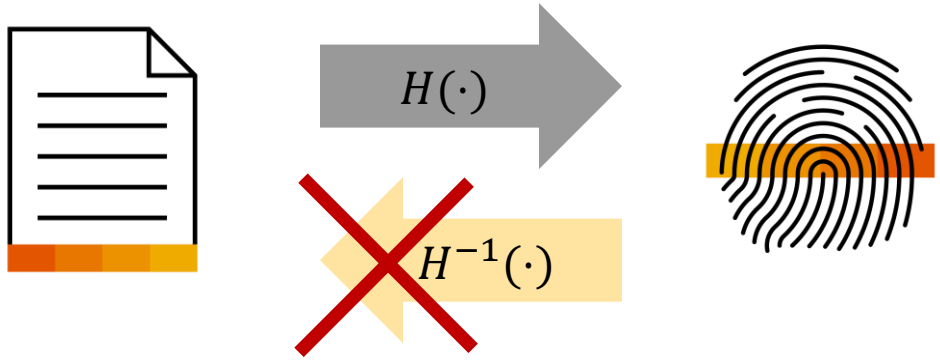
CTR Mode - Decryption



Hashes and MACs

Cryptographic Hash Function

Hash Function: Maps an input of arbitrary length to an output of fixed length



Cryptographic: Hard to invert

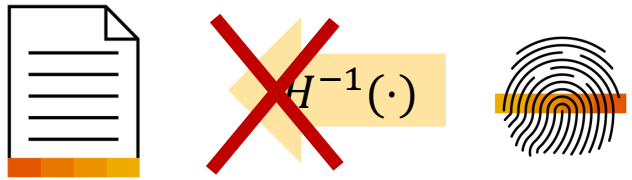
- $x \rightarrow H(x)$: easy (to compute)
- $H(x) \rightarrow x$: almost impossible

Since multiple inputs are mapped to the same output, „collisions“ are possible.

Properties of Cryptographic Hash Functions

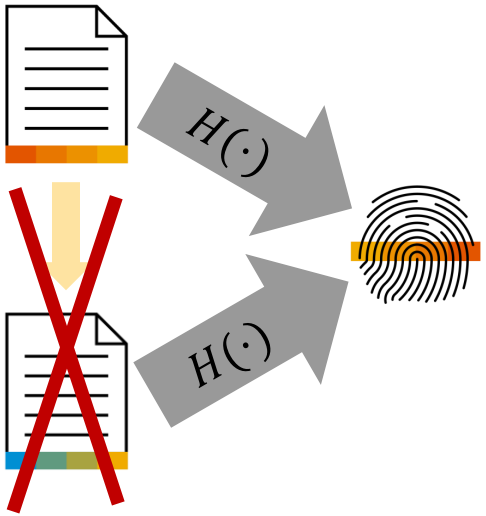
Preimage-resistant

- Given H(x) it is hard to find an (arbitrary) x



Second preimage-resistant

- Given x it is hard to find x‘ (x‘ ≠ x), such that H(x) = H(x‘)



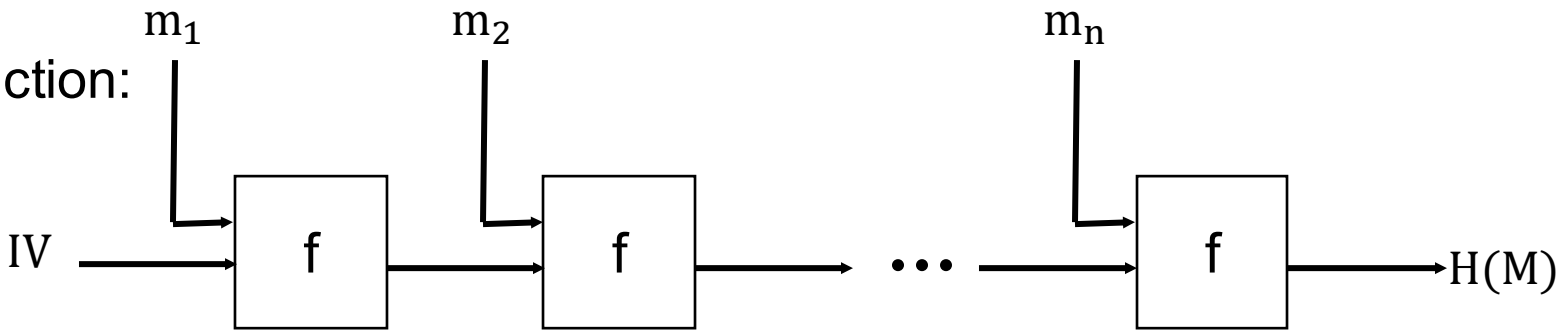
Collision-resistant

- It is hard to find any x and x‘, such that H(x) = H(x‘)

Examples

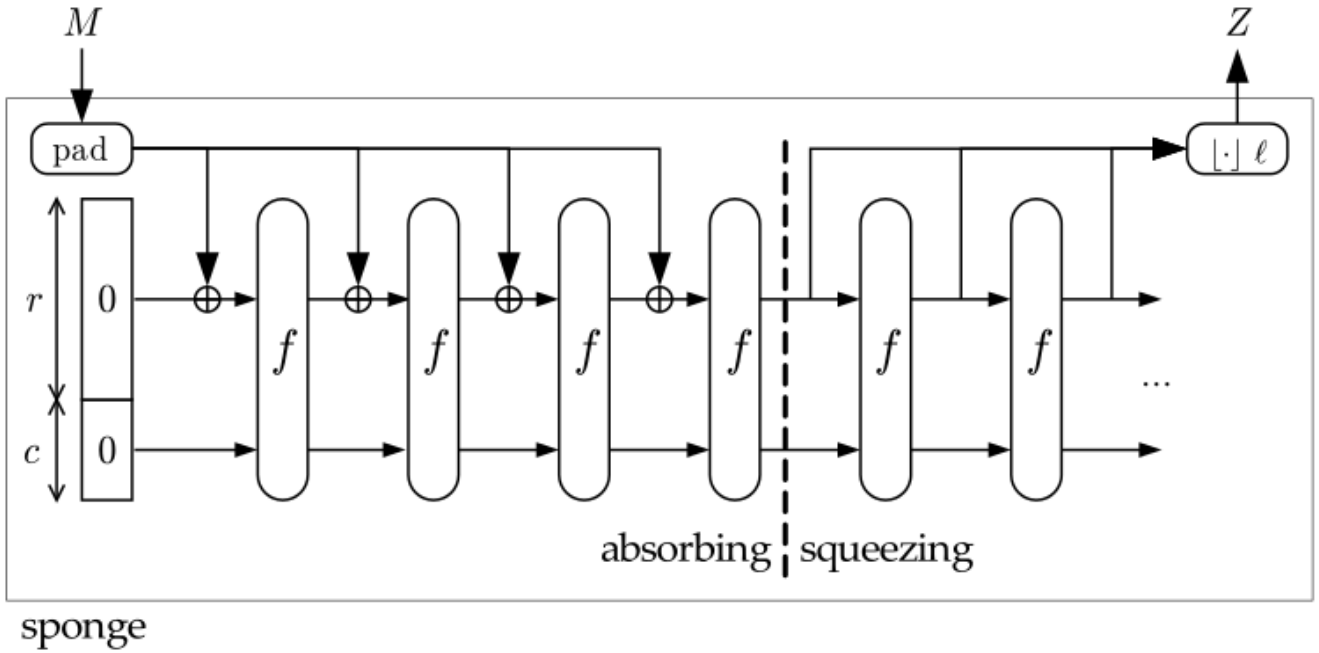
Based on Merkle Damgard Construction:

- MD5: 128 bit (Collisions found)
- SHA-1: 160 bit (Collisions found)
- SHA-2: SHA-256/512 256/512 bit



Based on Sponge / Keccak Construction:

- SHA-3



From: https://keccak.team/sponge_duplex.html

First SHA-1 Collision

Published on 23 February 2017 by Google and CWI Amsterdam - https://shattered.io/

~100.000 times better than brute force

- 9,223,372,036,854,775,808 SHA1 operations
- 6500 years of computation for a single CPU
- 110 years of computation for a single GPU
- Costs between 75.000$ and120.000$ on Amazon EC2

First chosen-prefix collision for SHA-1

Published on 07 January 2020 by Gaëtan Leurent and Thomas Peyrin - https://sha-mbles.github.io/

- Chosen prefix collision under 2 months using 900 Nvidia GTX 1060 GPUs

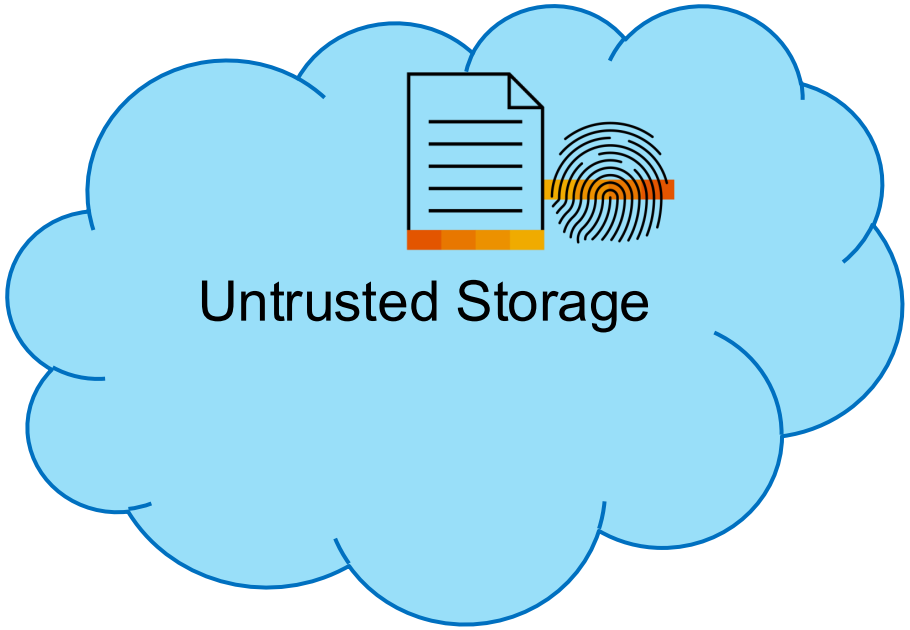
~450,000 times better than brute force ($2^{61.2}$)

- 27 years using single GTX 1060 GPU
- Costs ~11k US$

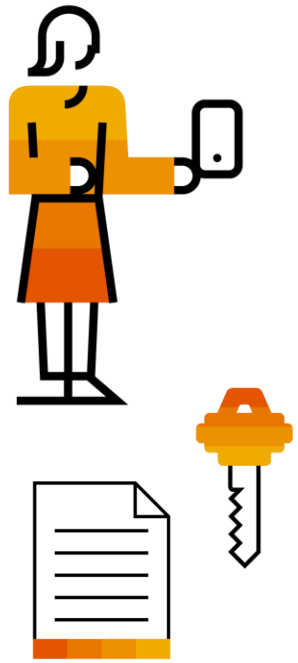
~100,000 times better than brute force ($2^{63.4}$) for chosen prefix

- 107 years using single GTX 1060 GPU
- Costs ~45k US$

Hash Functions



Message Authentication Codes



„Hash function with key“

Computation-resistant

- Given arbitrary pairs x, MAC(x, k) it is hard to compute x‘, MAC(x‘, k)

Message Authentication Codes

Key Generation:

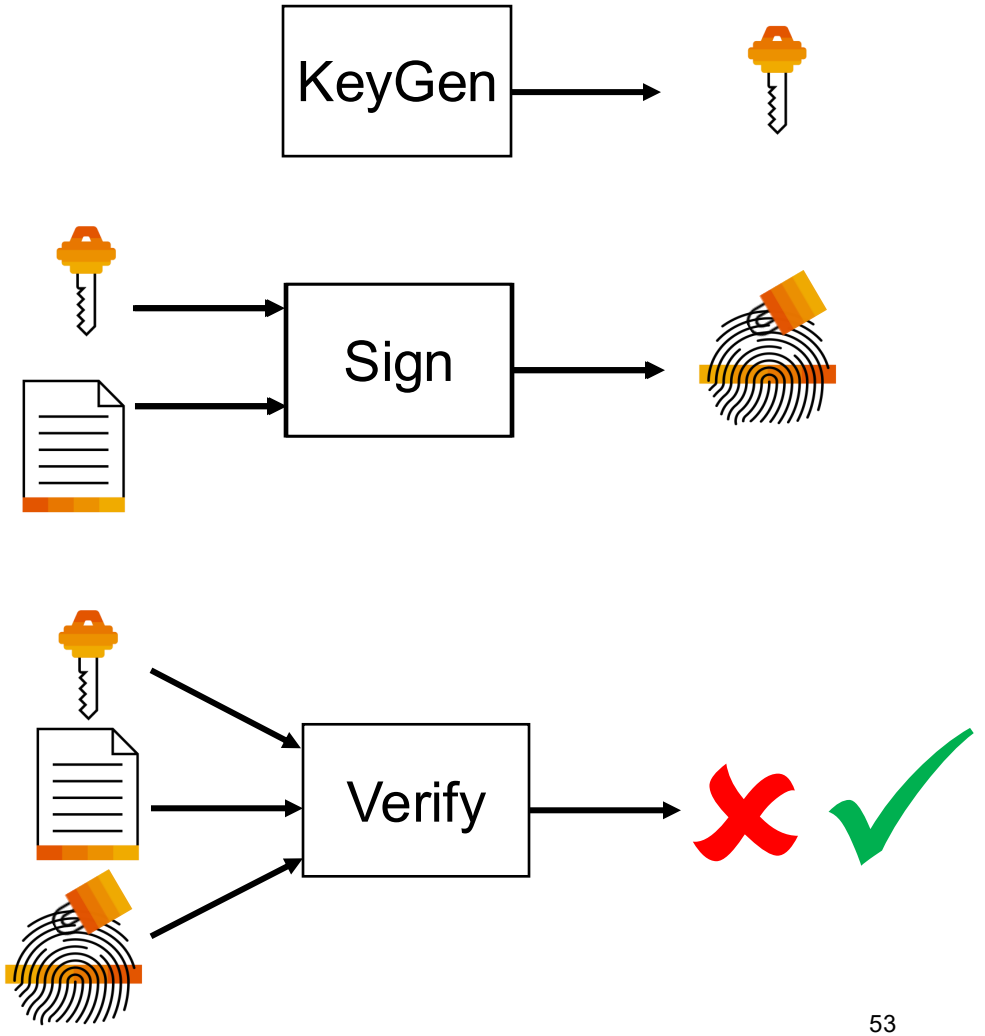
- Generates a key used for signing and verifying.

Sign:

- Given the key and a message outputs a MAC.

Verify:

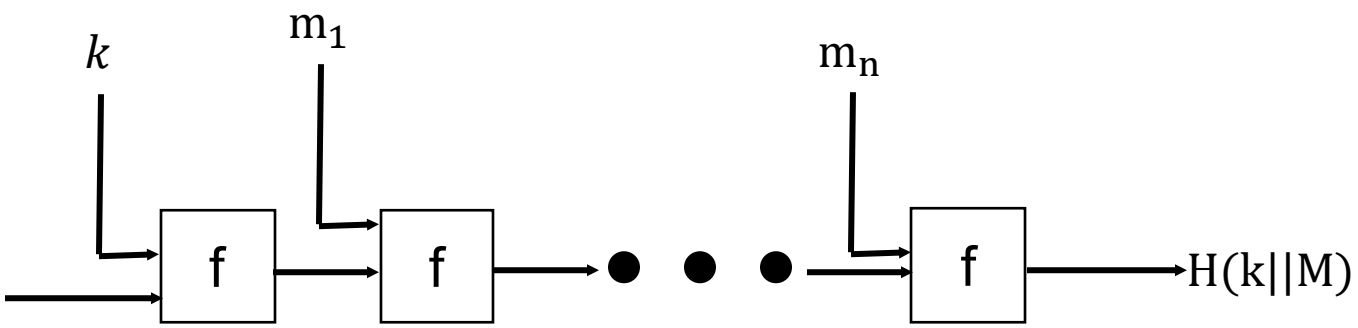
- Given the key, the MAC and the message accepts or rejects the MAC.

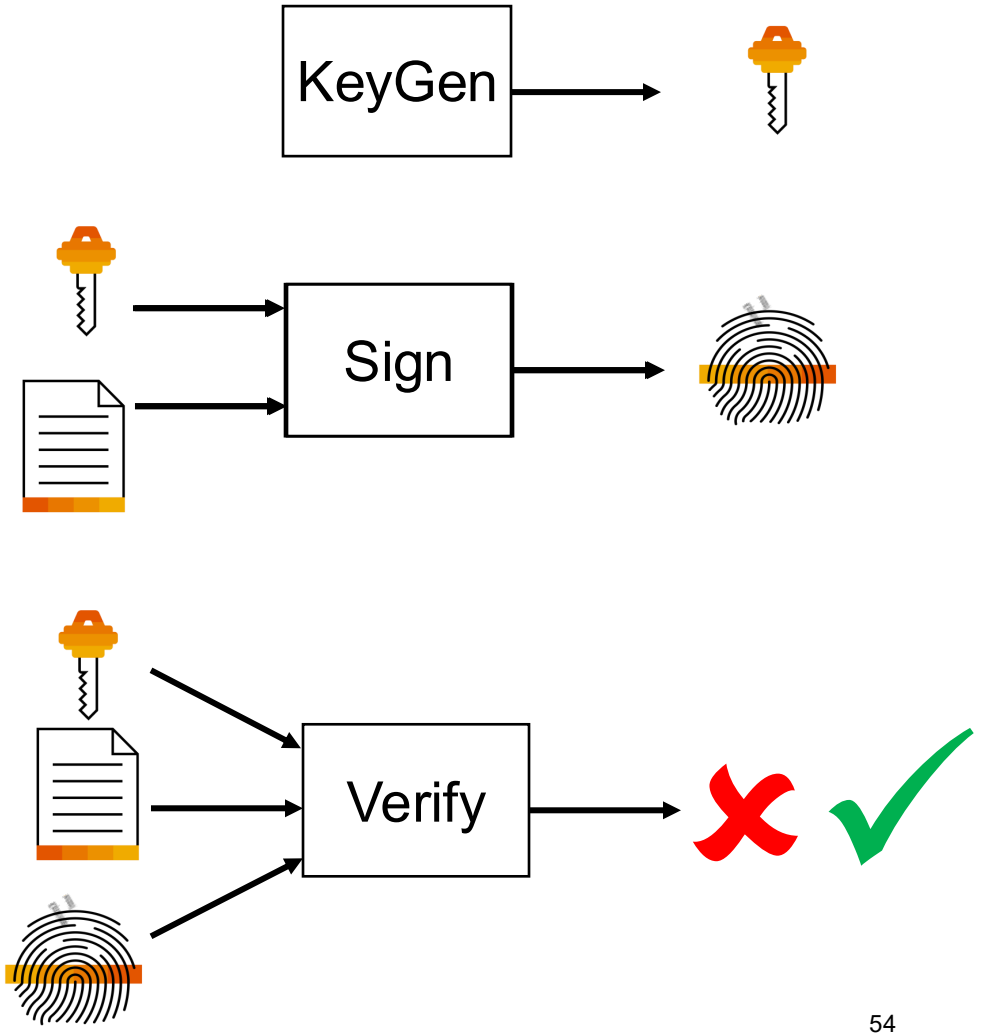


HMAC – *What can go wrong?*

HMAC: Using a hash function H, a key k and message m

- Never use the construction H(k, M)

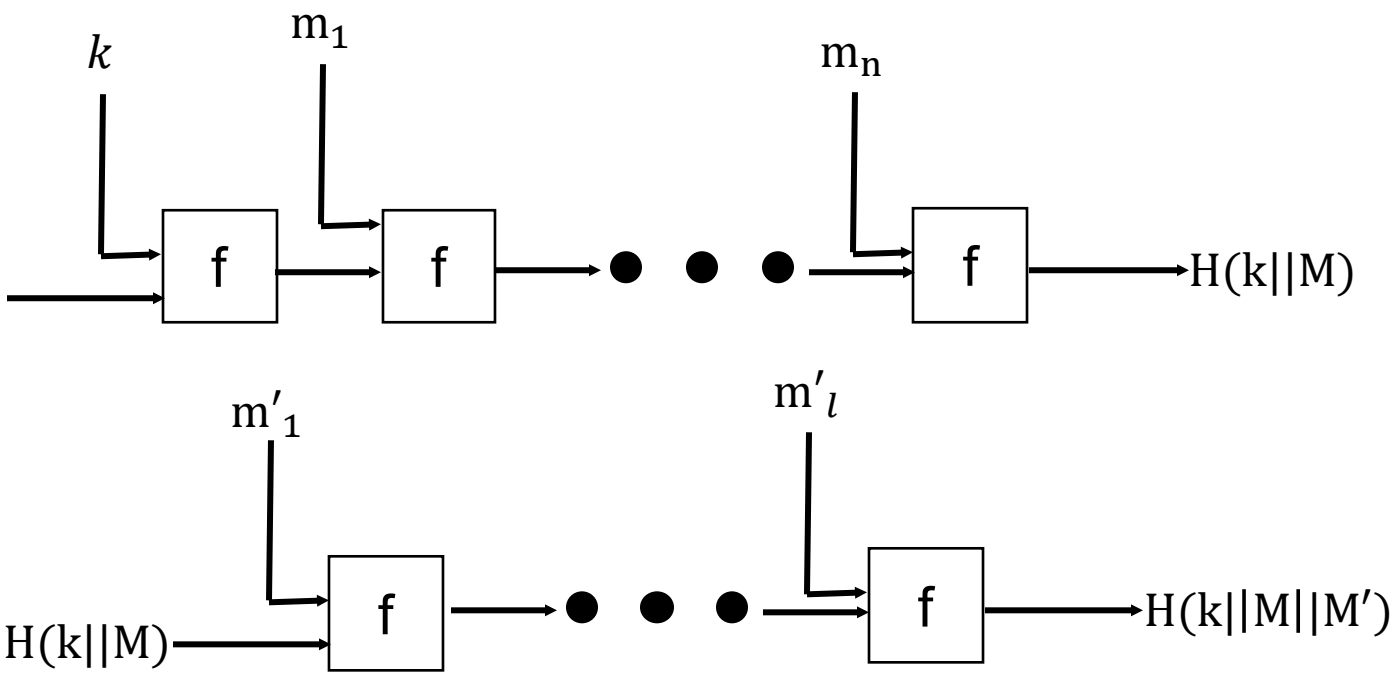


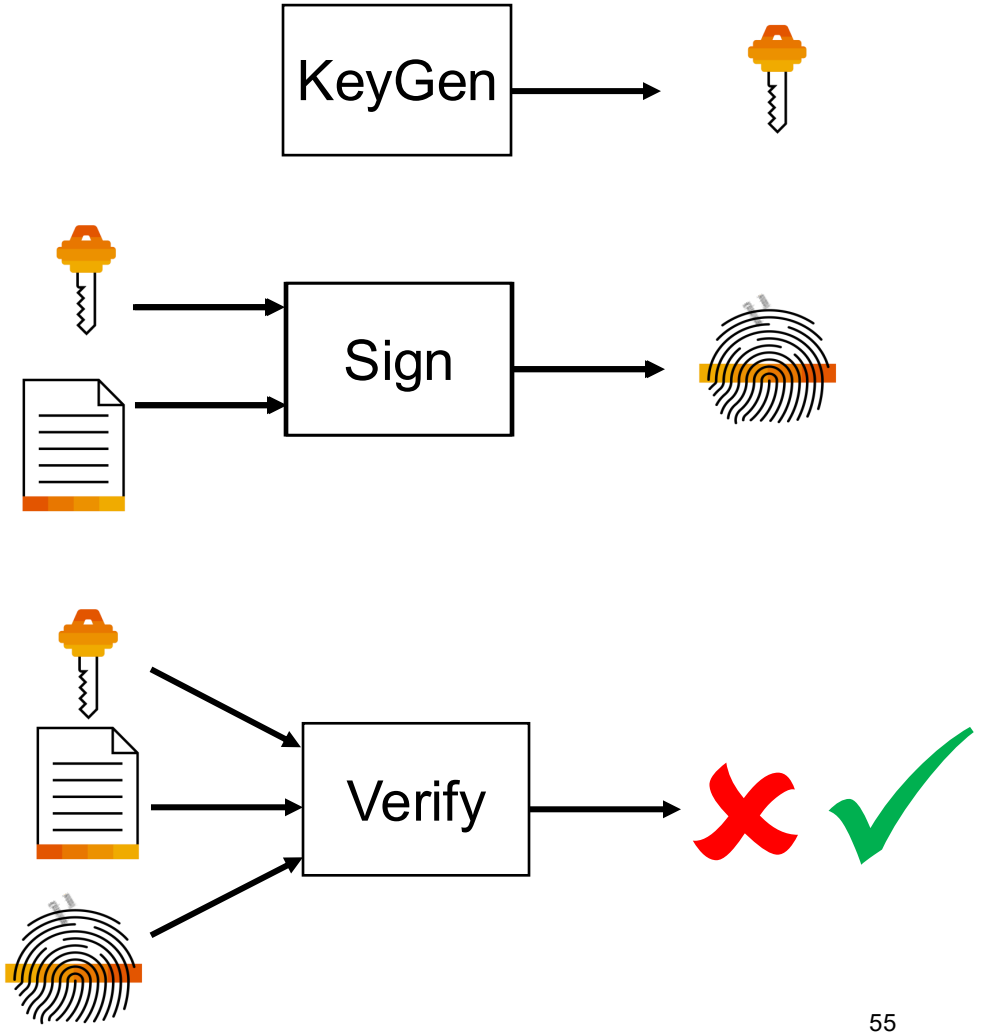


HMAC – *What can go wrong?*

HMAC: Using a hash function H, a key k and message m

- Never use the construction H(k, M)



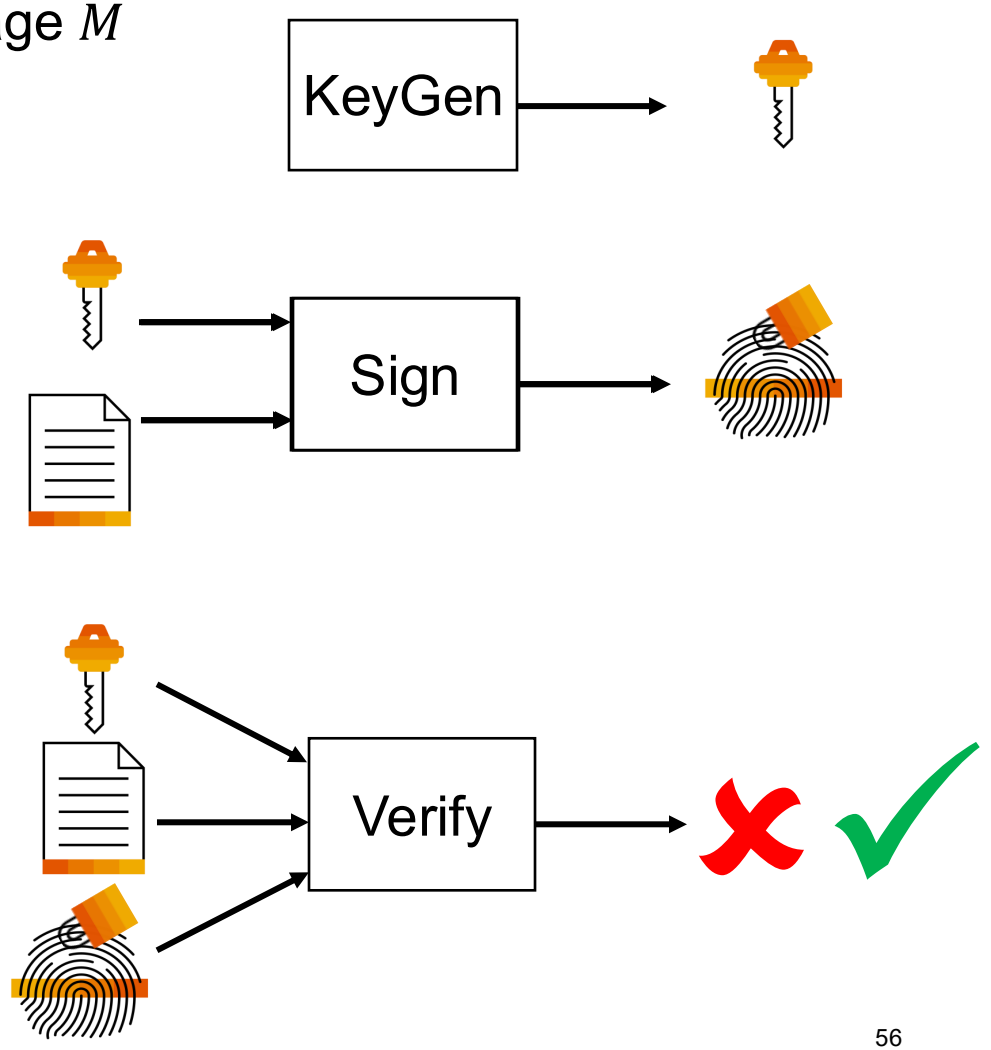


HMAC – *How to do it right?*
RFC 2104: HMAC: Keyed-Hashing for Message Authentication

HMAC: Using a hash function H, a key k, padding and message M

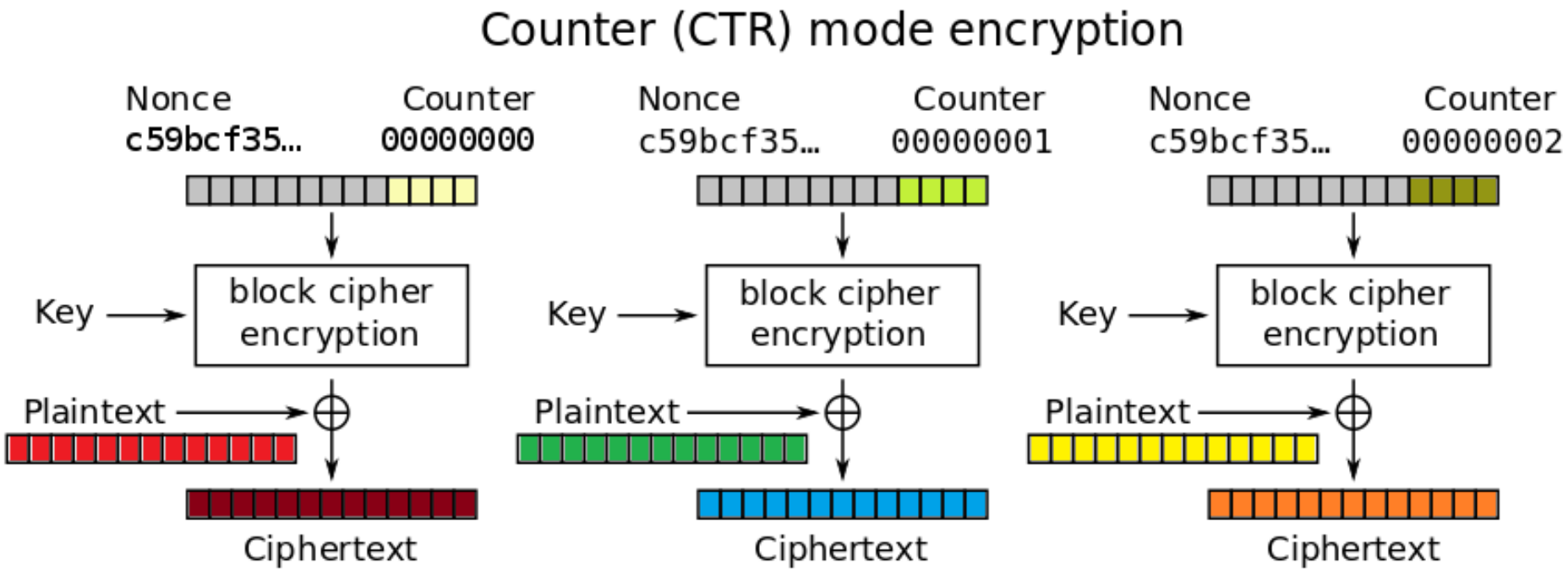
H(k || *opad* || H(k || *ipad* || M))

- With standardized padding values
  - *opad* is the outer padding (0x5c5c5c…5c5c)
  - *ipad* is the inner padding (0x363636…3636)



Galois/Counter Mode

Authentication via GCM



Remember this one?

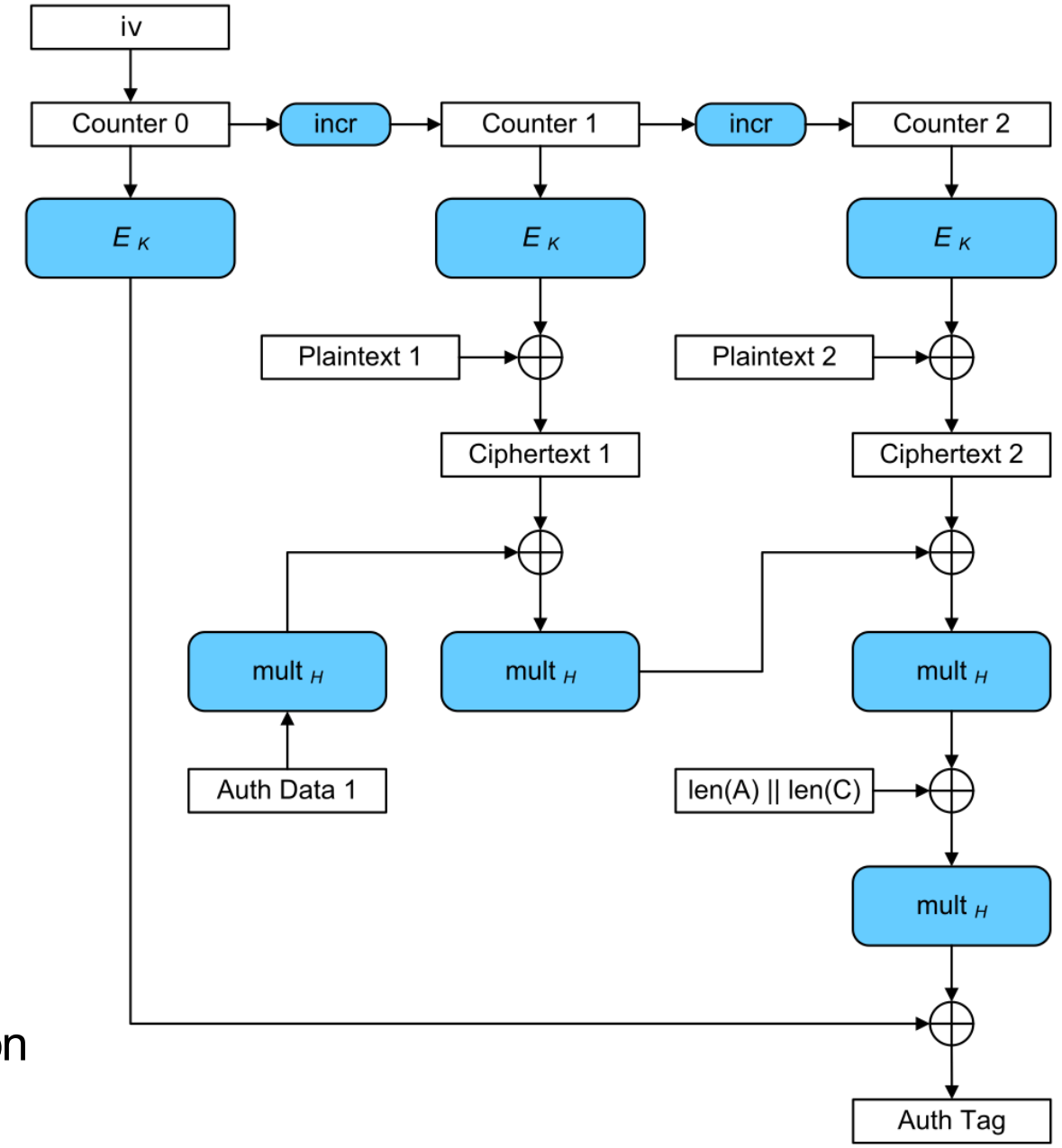
Authentication via GCM

GCM: Galois Counter Mode

Provides both:

- Confidentiality
- Integrity

Auth Code: Galois mode of authentication



Passwords and Key Derivation

Passwords and KDFs

Do not use ASCII String as cryptographic key!

Use dedicated Key Generation Function (offers sufficient entropy)

Ciphertext and keys should be stored separately

If at all, keys should not be stored in plaintext

- Password protected key store
- Hardware security modules (HSM)

Password Protected Key Store

Use Password-Based Key Derivation Function PBKDF2 to derive key from password

- **RFC 2898**: PKCS #5: Password-Based Cryptography Specification
- SAP Product Standard Security **SEC-266**
  https://wiki.wdf.sap.corp/wiki/display/PSSEC/SEC-266#SEC-266-Password-basedKeyDerivation(PBKDF2)

Use derived key to encrypt private/secret keys

Only as strong as password